Metaheuristic Approaches to Function Optimization: Hill Climbing and Simulated Annealing

Olivia-Alexandra Gavril

29 October 2025

Abstract

Searching for an optimal solution in a multidimensional landscape is like climbing through misty hills, because every ascent feels promising, until a higher peak emerges beyond the fog. Algorithms like Hill Climbing and Simulated Annealing were designed to make that climb smarter. Optimization is one of the most powerful tools in process integration. Optimization involves the selection of the "best" solution from among the set of candidate solutions. [14] In this report, we compare the performance of these two algorithms on four well-known benchmark functions: De Jong, Schwefel, Rastrigin, and Michalewicz. Each algorithm was tested on multiple dimensions to evaluate their efficiency, convergence behavior, and adaptability to complex search spaces. While Hill Climbing takes a cautious approach, refining solutions through incremental improvements, Simulated Annealing introduces controlled randomness, allowing occasional uphill moves that help escape local minima and explore the search space more broadly.

1 Introduction

Optimization is, at its core, about finding the best possible choice, such as the shortest route, the lowest cost, or the highest performance. In simple problems, this can be done by comparing a few alternatives. But as the number of variables grows, the search space becomes vast and complex, filled with many local optima that can easily mislead traditional methods.

To deal with this kind of complexity, researchers began turning to heuristic and metaheuristic algorithms, approaches that do not guarantee the perfect solution but can find very good ones in a reasonable time. These algorithms mimic adaptive behaviors found in nature or human problem solving, using exploration and gradual improvement to move through the search space.Two of the most representative examples are Hill Climbing and Simulated Annealing. Both start from a random point in the landscape and iteratively improve upon it.

Even though they are conceptually similar, these two algorithms behave quite differently. Hill Climbing follows a deterministic and local search strategy, but it can take several forms depending on how new solutions are selected. In the Best Improvement variant, all neighbors of the current solution are evaluated, and the algorithm moves to the one that offers the largest improvement. The First Improvement version, on the other hand, stops as soon as it finds a better neighbor, which makes it faster but less exhaustive. The Worst Improvement variant explores the opposite idea, because it accepts the least favorable improvement among better neighbors.

Simulated Annealing, in contrast, introduces a probabilistic acceptance rule inspired by the physical process of annealing in metals. By occasionally accepting worse solutions according to a temperature dependent probability, it helps the search escape local minima and increases the chances of reaching a better global optimum.

In this report, we compare the performance of these two algorithms on four benchmark functions that are commonly used to evaluate optimization methods: De Jong, Schwefel, Rastrigin and Michalewicz. These functions present different levels of difficulty, starting from simple convex landscapes to highly multimodal ones with many local minima. The main objective is to analyze how each algorithm performs depending on the type of function and the problem's dimensionality. We also look at the influence of key parameters, such as the neighborhood definition in Hill Climbing and the cooling schedule in Simulated Annealing.

2 Methods and Implementation

Solution representation

In the context of the Hill Climbing and Simulated Annealing algorithms , each candidate solution is represented as a bitstring. Since the benchmark functions (De Jong, Schwefel, Rastrigin, Michalewicz) take as input an array of real numbers, it is necessary to encode these values into a discrete form, suitable for binary represenation.
The search space is divided according to a chosen precision, denoted as 10^{-d} . Each variable's interval *[a, b]* is split into equal segments of this size, which determines the number of bits needed to represent all possible values, computed as:

$$N = (b - a) \times 10^d$$

The binary encodings of all variables are concatenated to form a single bitstring representing the entire solution. Consequently, each candidate solution is mantained as one continuous sequence of bits rather than separate binary vectors.
Before evaluating a solution with the optimisation function, every binary-encoded variable must be converted back into its real form using the formula:

$$X_{real} = a + \frac{decimal(x_{bits}) \times (b - a)}{2^n - 1}$$

This encoding allows the algorithms to uniformly sample and evaluate the search space. Each bitstring uniquely defines a candidate solution, while de decoding equation rescales the binary value to its real counterpart within *[a, b]*, making it suitable for function evaluation.

Random Initialization with Mersenne Twister

Each optimisation algorithm begins from an initial random solution, which serves as the starting point for the search process. This initial candidate is generated using the Mersenne Twister *('mt19937')*, a well-established pseudorandom number generator [13]. For statistical robustness, in each run of the algorithm it is used a different random seed, ensuring independent initialization while peserving uniform randomness through the Mersenne Twister generator.

Hill Climbing algorithm

Hill climbing is a heuristic search algorithm that belongs to the family of local search methods [9]. It operates by iteratively exploring the neighborhood of the current solution and selecting moves that makes an improvement according to an evaluation or fitness function. The process begins with a randomly generated solution (initial state) and proceeds by examining neighboring solutions, which are then evaluated to determine whether an improvement has been achieved. The new candidate solution is created by slightly modifying the current one: in this case, by flipping one bit the bitstring that represents the solution. The improvement strategy is determined by the chosen *Hill Climbing variant*: First, Best or Worst Improvement.
For the *First Improvement* Hill Climbing, the neighboring solutions are evaluated sequentially until a solution offering an improvement over the current one is found. The algorithm then moves to this solution without examining the remaining neighbors, reducing computational cost but potentially missing larger improvements. The *Best Improvement* Hill Climbing, also known as the Steepest-Ascent Hill Climbing, performs a complete evaluation of all the neighboring states at each step and moves towards the one offering the highest improvement in the optimization function chosen. This exhaustive search ensures a more consistent progress towards the optimum[9]. In The *Worst Improvement* Hill Climbing method, the next state is chosen as the neighbor that offers the least improvement, while still being better than the current solution. The chance of escaping local optima is can increase with this method of controlled progress. Once a neighboring solution that improves the objective function is identified, the algorithm transitions to that state and resumes the search. The process continues until no further improvement can be obtained from the neighboring solutions. After that, the obtain local minimum is compared with the best solution identified so far, and if it is better, it replaces it. These steps are repeated until the maximum number of iteration is reached. To avoid stagnation and help the algorithm escape local minima, a random restart was introduced whenever no improvement was observed for several iterations.

Simulated Annealing algorithm

Simulated Annealing is a probabilistic optimization algorithm inspired by the physical process of annealing in metallurgy: when a material is heated to a high temperature and then slowly cooled to reach a stable structure with minimal energy. In optimization, this analogy translates to exploring a solution space to find the global minimum of an objective function[2]. The behavior of the Simulated Annealing algorithm is mainly determined by a set of control parameters that guide its search dynamics. The most important among these are the *initial temperature*(T_0), *the cooling rate*(α) and *the number of iterations* performed at each temperature level. The initial temperature T_0 was estimated using a short calibration phase, during which several random transitions are generated from the initial solution. For each transition, the change in the objective function Δf is computed. The average of all positive changes $\overline{\Delta f^+}$ is then used in the formula:

$$T_0 = \frac{\overline{\Delta f^+}}{\ln\left(\frac{m_2}{m_2 p_0 - m_1(1-p_0)}\right)}$$

[2]

where:

- m_1 – number of improving transitions ($\Delta f \leq 0$)
- m_2 – number of worsening transitions ($\Delta f > 0$)
- p_0 – initial acceptance rate (defined by the user)
- $\overline{\Delta f^+}$ – mean increase in the objective function for worsening transitions [2]

At the start of the process, more worse solutions are accepted, which helps the algorithm explore more freely before gradually focusing on better areas. The initial temperature defines a probability of accepting worse solutions at the beginning of the process, according to the *Metropolis acceptance criterion*, defined as $P = e^{-\frac{\Delta f}{T}}$, where Δf represents the change in the objective function. This rule allows Simulated Annealing algorithm to escape local minima by accepting non-improving values with a probability that decreases as the temperature T is reduced. The temperature follows a geometric cooling scheme, expressed as $T_{k+1} = \alpha T_k$, where $0.8 < \alpha < 0.99$. [2] Higher values of α lead to a slower cooling process, encouraging broader exploration of the search space at the cost of increased computation time. At each temperature level, a number of neighboring solutions are generated and evaluated. The algorithm stops when the temperature reaches a predefined number of iterations. In this implementation, new candidate solutions are generated by flipping one or more bits in the binary representation of the current solution. These parameters must be selected carefully to maintain a good balance between exploring new regions of the search space and improving existing solutions. Cooling too quickly can cause the algorithm to stop exploring too soon, while cooling too slowly increases computation time.

3 Experiments and Results

In this section, there are presented the statistical results obtained from optimizing four benchmark functions (De Jong, Schwefel, Rastrigin, and Michalewicz) using the Hill Climbing and Simulated Annealing algorithms. The experiments aim to evaluate and compare the performance, accuracy and efficiency of the two optimization methods across different problem dimensions.

Experimental Setup

In order to explore the behavior of the benchmark functions, *three problem dimensions* were selected: 5, 10 and 30.For each dimension, the *Hill Climbing* algorithm and its variants (First Improvement, Best Improvement and Worst Improvement) were executed for *1,000 iterations*, while the *Simulated Annealing* algorithm was run for *10,0000 iterations*. This difference in iteration counts allowed for more precise statistical results, adjusted for the characteristics of each algorithm. The entire procedure was repeated *30 times* to ensure result consistency.

3.1 De Jong's Function

So called *first function of De Jong's* is one of the simplest test benchmark. [12] The function is continuous, convex and unimodal. General function definition and restrictions:

$$f_1(\mathbf{x}) = \sum_{i=1}^{n} x_i^2, \qquad -5.12 \leq x_i \leq 5.12 \tag{1}$$

Global minimum $f(x) = 0$ is obtainable for $x_i = 0$, where $i = 1, \ldots, n$.

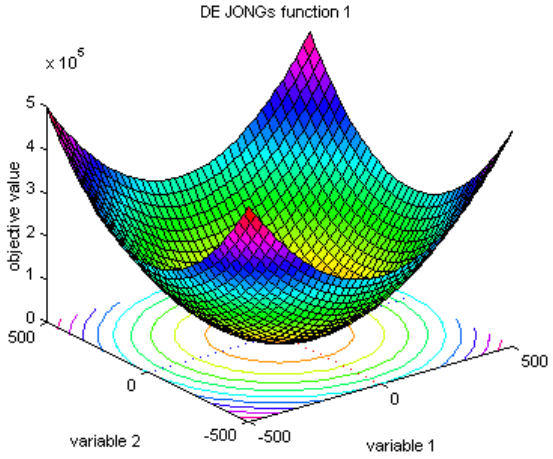

Figure 1: Visualisation of De Jong's function, with a surface plot for variables in the range [-500, 500].[3]

Considering the landscape of the De Jong's function, it is reasonable to expect that the local search algorithms will quickly converge towards the global minimum, without getting trapped in any local optima. The results produced by the optimization algorithms are summarized in the following tables.

Table 1: Comparison of optimization results for De Jong's function

	Dimension = 5			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	0.00000	0.00000	0.00000	0.00000
Hill Climbing: Best Improvement	0.00000	0.00000	0.00000	0.00000
Hill Climbing: Worst Improvement	0.00000	0.00000	0.00000	0.00000
Simulated Annealing	0.00000	0.04287	0.00849	0.01446

	Dimension = 10			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	0.00000	0.00000	0.00000	0.00000
Hill Climbing: Best Improvement	0.00000	0.00000	0.00000	0.00000
Hill Climbing: Worst Improvement	0.00000	0.00000	0.00000	0.00000
Simulated Annealing	0.00000	0.04723	0.00319	0.00970

	Dimension = 30			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	0.00000	0.00000	0.00000	0.00000
Hill Climbing: Best Improvement	0.00000	0.00000	0.00000	0.00000
Hill Climbing: Worst Improvement	0.00000	0.00000	0.00000	0.00000
Simulated Annealing	0.00000	0.00001	0.00000	0.00000

Table 2: Comparison of **running time** of the optimization algorithms for De Jong's function, **in seconds** (not mentioned) and **minutes**

	Dimension = 5		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	0.57232	2.06806	0.97773 s
Hill Climbing: Best Improvement	0.56435	2.49474	1.13266
Hill Climbing: Worst Improvement	0.61399	2.70026	1.14671
Simulated Annealing	0.18225	1.64633	0.78306

	Dimension = 10		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	2.19013	14.62527	4.39601
Hill Climbing: Best Improvement	2.27127	8.26781	4.03072
Hill Climbing: Worst Improvement	2.39337	9.47413	4.34086
Simulated Annealing	0.84238	3.00025	2.21185

	Dimension = 30		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	25.91104	85.49842	42.85757
Hill Climbing: Best Improvement	0.4774 min	2.21025 min	0.90836 min
Hill Climbing: Worst Improvement	0.54699 min	2.13076 min	1.04446 min
Simulated Annealing	13.48448	26.84616	23.77441

The results obtained from the algorithms were, as expected, satisfactory. Both Hill Climbing and Simulated Annealing successfully converged to the global minimum of the De Jong's function. Although higher dimensions increased the computational time, the execution time was relatively low, reflecting the simplicity of the function's landscape.

3.2 Schwefel's Function

Schwefel's function is particularly deceptive because its global minimum is located far away from most local minima, making it challenging for optimization algorithms to locate the true optimum. The function has the following definition and restrictions:

$$f_7(\mathbf{x}) = \sum_{i=1}^{n} -x_i \cdot \sin\left(\sqrt{|x_i|}\right), \qquad -500 \leq x_i \leq 500 \tag{2}$$

Its global minimum $f(x) = -418.9829n$ is obtainable for $x_i = 420.9687$, where $i = 1, \ldots, n$.

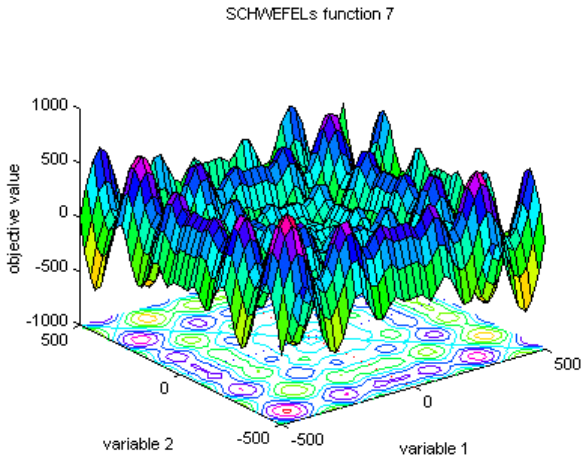

Figure 2: Visualization of Schwefel's function with a surface plot for variables in the range [-500, 500]. [3]

As the global minimum of the Schwefel function is surrounded by numerous local minima, it is expected that the Hill Climbing algorithm will eventually get trapped in one of them. However, due to the optimizations applied to the algorithm, it performed surprisingly well and managed to reach near-optimal solutions in some cases. In contrast, Simulated Annealing is better suited for this type of landscape, as its probabilistic nature allows it to occasionally escape local traps and continue exploring.

Table 3: Comparison of optimization results for Schwefel's function

	Dimension = 5			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	-1895.321	-1142.963	-1541.724	186.865
Hill Climbing: Best Improvement	-2094.707	-1263.014	-1691.276	197.6043
Hill Climbing: Worst Improvement	-2006.809	-1230.29	-1544.643	174.5579
Simulated Annealing	-2094.913	-2094.496	-2094.712	0.09906

	Dimension = 10			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	-3561.499	-2736.975	-3135.864	208.3373
Hill Climbing: Best Improvement	-3941.593	-2825.47	-3307.447	267.5968
Hill Climbing: Worst Improvement	-3617.15	-2676.319	-3179.849	249.318
Simulated Annealing	-4189.619	-4188.999	-4189.359	0.15066

	Dimension = 30			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	-10656.63	-8276.408	-9355.231	549.099
Hill Climbing: Best Improvement	-11381.79	-9062.042	-10091.11	527.2629
Hill Climbing: Worst Improvement	-10244.87	-8641.939	-9292.544	436.5778
Simulated Annealing	-12568.44	-12166	-12413.02	126.4578

For comparison, the theoretical global minimum values for each dimensionality are as follows: for $\mathbf{n = 5}$, f(x) = -2094.9145; for $\mathbf{n = 10}$, f(x) = -4189.829; for $\mathbf{n = 30}$; f(x) = -12,569.487.

Table 4: Comparison of **running time** of the optimization algorithms for Schwefel's function, **in seconds**(not mentioned) and minutes.

	Dimension = 5		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	1.02335	4.6458	1.58705
Hill Climbing: Best Improvement	1.12361	4.60389	1.91616
Hill Climbing: Worst Improvement	1.26237	8.87698	2.49018
Simulated Annealing	0.43139	3.88785	2.31029

	Dimension = 10		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	3.90911	17.95018	6.40102
Hill Climbing: Best Improvement	4.70103	13.41338	7.24898
Hill Climbing: Worst Improvement	6.068	31.07414	13.42879
Simulated Annealing	0.38667	3.08434	2.13053

	Dimension = 30		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	0.63506 min	2.71033 min	0.95445 min
Hill Climbing: Best Improvement	0.7755 min	14.8691 min	1.74354 min
Hill Climbing: Worst Improvement	1.95872 min	6.02793 min	3.50056 min
Simulated Annealing	17.06229	18.47662	17.85493

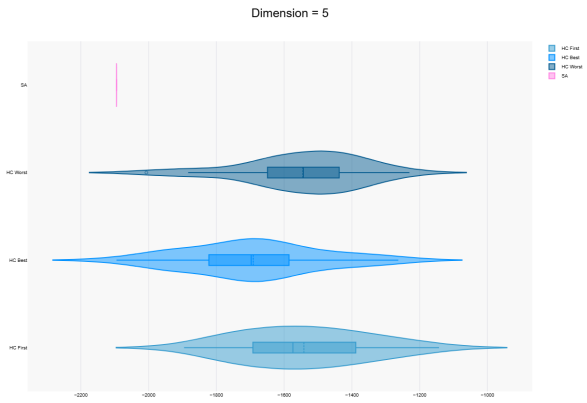

Figure 3: Violin plot showing the distribution of minimum values for the Schwefel function in 5 dimensions across all algorithms. [17]

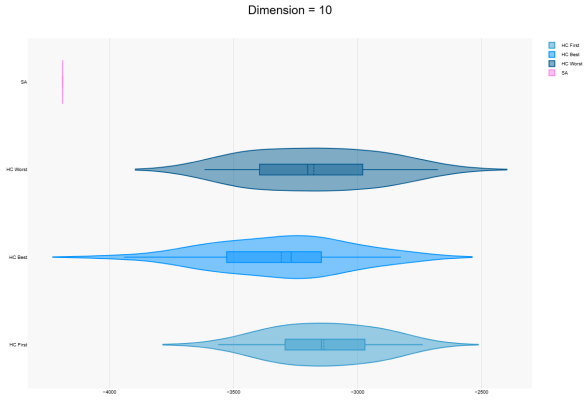

Figure 4: Violin plot showing the distribution of minimum values for the Schwefel function in 10 dimensions across all algorithms. [17]

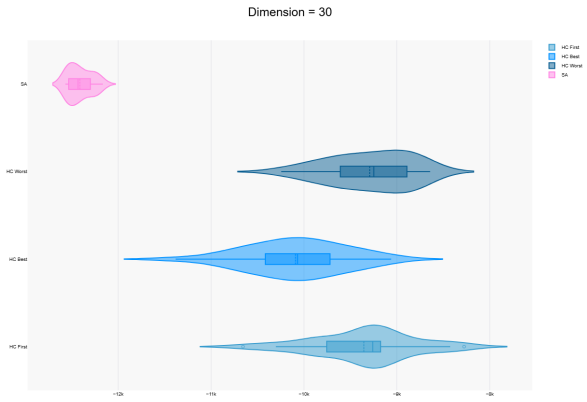

Figure 5: Violin plot showing the distribution of minimum values for the Schwefel function in 30 dimensions across all algorithms. [17]

For all dimensions, the violin plots ,beside the tables, reveal clear behavioral patterns for the optimization algorithms.The plots show that Hill Climbing performs well on simpler problems but becomes less reliable as dimensionality increases. Among its variants, the Best Improvement version achieves the

most stable results. Simulated Annealing, on the other hand, demonstrates stable performance across all test cases. The violin plots show that it consistently finds solutions closer to the global minimum, with minimal variance even as the problem dimension grows.

3.3 Rastrigin's Function

Rastrigin's function is based on the function of De Jong with the addition of cosine modulation in order to produce frequent local minima. Therefore, the test function is highly multimodal. However, the location of the minima are regularly distributed.[3] The function has the following definition and constrains:

$$f_6(\mathbf{x}) = 10 \cdot n + \sum_{i=1}^{n} \left(x_i^2 - 10 \cdot \cos(2\pi x_i)\right), \qquad -5.12 \leq x_i \leq 5.12 \tag{3}$$

Global minimum $f(x) = 0$ is obtainable for $x_i = 0$, where $i = 1, \ldots, n$.

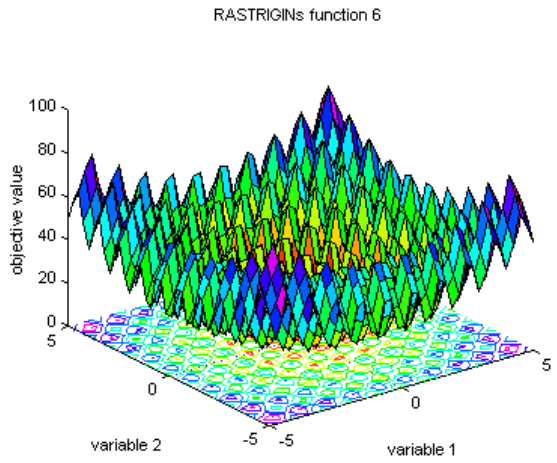

Figure 6: Visualization of Rastrigin's function left with a surface plot for variables in the range [-5 , 5] [3]

Table 5: Comparison of optimization results for Rastrigin's Function

	Dimension = 5			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	3.46661	26.98438	14.31798	6.08346
Hill Climbing: Best Improvement	3.4666	15.98912	9.89137	3.08407
Hill Climbing: Worst Improvement	4.99989	37.24461	14.454	6.46993
Simulated Annealing	0.00000	10.89486	3.74039	3.77049

	Dimension = 10			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	10.13907	36.06353	23.01735	6.36207
Hill Climbing: Best Improvement	7.69236	37.92422	19.66993	6.59668
Hill Climbing: Worst Improvement	17.15856	35.85021	26.06891	4.34619
Simulated Annealing	0.00000	11.68616	2.49269	3.09591

	Dimension = 30			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	48.82213	92.75589	71.58258	12.23673
Hill Climbing: Best Improvement	36.21646	86.43961	60.52116	11.71494
Hill Climbing: Worst Improvement	61.86926	109.4828	83.03921	3.11524
Simulated Annealing	3.70748	22.16531	11.84847	12.03842

Table 6: Comparison of **execution time** of the optimization algorithms for Rastrigin's function

	Dimension = 5		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	0.70251	2.6018	1.13043
Hill Climbing: Best Improvement	0.78214	5.13747	1.44105
Hill Climbing: Worst Improvement	0.77224	2.62908	1.13578
Simulated Annealing	0.43139	3.88785	2.31029

	Dimension = 10		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	2.46603	7.09086	3.81312
Hill Climbing: Best Improvement	0.78214	5.13747	0.38417
Hill Climbing: Worst Improvement	1.02235 min	2.40717 min	1.31868 min
Simulated Annealing	17.06229	18.47662	17.85493

	Dimension = 30		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	28.39111	75.33025	44.12126
Hill Climbing: Best Improvement	0.66357 min	28.30065 min	1.93763 min
Hill Climbing: Worst Improvement	1.02235 min	2.40717 min	1.31868 min
Simulated Annealing	13.48448	26.84616	23.77441

The results for the *Rastrigin* function highlight how demanding this landscape is due to its many evenly spaced local minima. As the dimension grows, the search space becomes more complex, making it harder for the algorithms to locate the global minimum.
The violin plots clearly reflect this behavior. Hill Climbing displays a broader variation in results, particularly as the dimensionality increases. This indicates its tendency to get "trapped". In contrast, Simulated Annealing produces more consistent and generally lower minima across runs, highlighting its ability to better explore the search space.

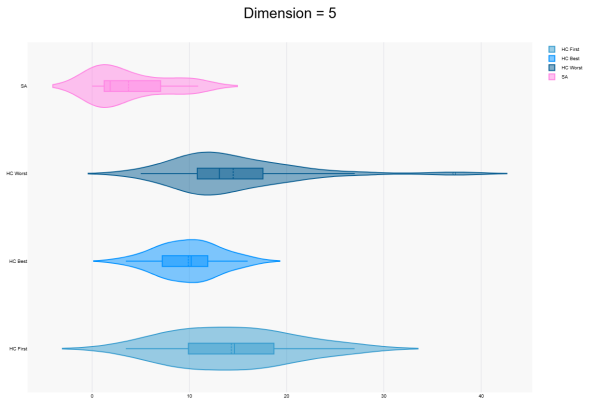

Figure 7: Distribution of minimum values obtained by each algorithm on the Rastrigin function with dimension 5.

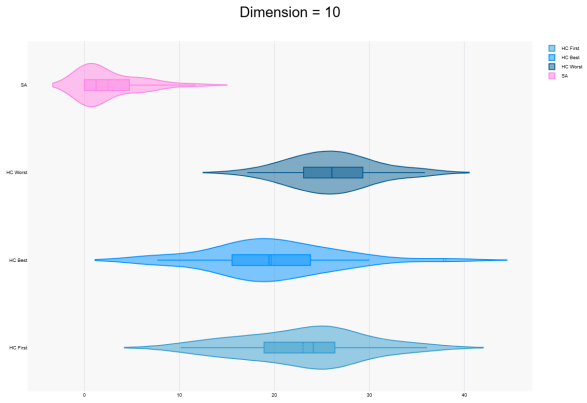

Figure 8: Distribution of minimum values obtained by each algorithm on the Rastrigin function with dimension 10.

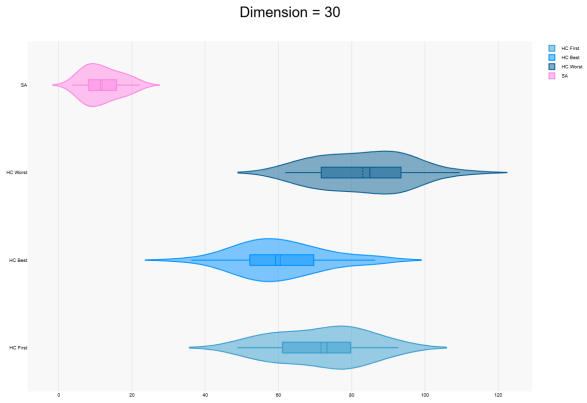

Figure 9: Distribution of minimum values obtained by each algorithm on the Rastrigin function with dimension 30.

3.4 Michalewicz's Function

The Michalewicz function is a multimodal benchmark function that possesses $n!$ local optima. [3] The parameter m controls the steepness of the valleys and ridges: higher values of m make the optimization problem increasingly difficult. For large m values, the function exhibits "needle-in-a-haystack" behavior, where most points in the search space provide little information about the location of the global optimum. [12]

$$f_{12}(\mathbf{x}) = -\sum_{i=1}^{n} \sin(x_i) \left[\sin\left(\frac{i\, x_i^2}{\pi} \right) \right]^{2m}, \qquad i = 1, \ldots, n,\ m = 10,\ 0 \le x_i \le \pi \tag{4}$$

The global minimum value was approximated as $f(x) = -4.687$ for $n = 5$ and $f(x) = -9.66$ for $n = 10$. The corresponding optimal solutions were not provided. [12]

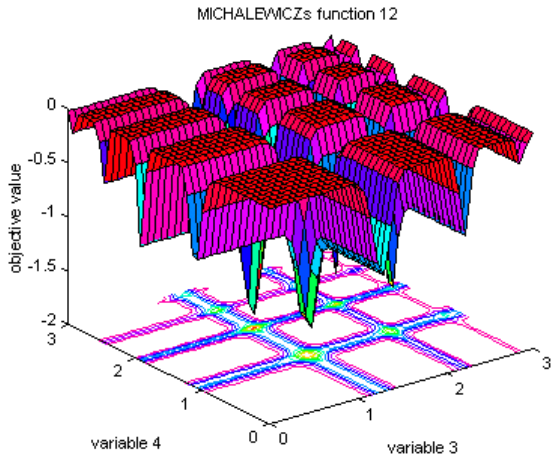

Figure 10: Visualization of the Michalewicz's function. Surface plot for variables x_3 and x_4 over the interval $[0, 3]$, and surface plot for variables x_1 and x_2 over the interval $[0, 3]$. [3]

Despite being a highly multimodal function, Hill Climbing performed surprisingly well on the Michalewicz function. This can be explained by the smooth and well-defined valleys in the function's shape, which "guide" the algorithm toward deep local minima. Additionally, the limited search space $[0, \pi]$allows the algorithm to explore efficiently without needing a wide global search. When combined with random restarts, the algorithm can escape shallow traps and often reach solutions that are very close to the global minimum.

Table 7: Comparison of optimization results for the Michalewicz's function

	Dimension = 5			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	-4.67025	-2.79532	-3.65719	0.56774
Hill Climbing: Best Improvement	-4.53502	-2.98869	-3.92992	0.49091
Hill Climbing: Worst Improvement	-4.49154	-2.31665	-3.27735	0.58601
Simulated Annealing	-4.68766	-4.26747	-4.59811	0.10485

	Dimension = 10			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	-8.74452	-6.09279	-7.54098	0.81336
Hill Climbing: Best Improvement	-9.03804	-6.91331	-8.0361	0.61628
Hill Climbing: Worst Improvement	-7.67192	-4.47456	-6.25797	0.84421
Simulated Annealing	-8.66015	-6.21197	-8.33858	0.45715

	Dimension = 30			
	Min. Value	**Max. Value**	**Mean**	**Standard Deviation**
Hill Climbing: First Improvement	-25.03433	-19.88829	-22.44486	1.29029
Hill Climbing Best Improvement	-26.42404	-20.27971	-23.79517	1.38709
Hill Climbing: Worst Improvement	-22.17156	-16.54717	-19.01304	1.31556
Simulated Annealing	-27.8186	-26.11362	-27.09268	0.47485

Table 8: Comparison of **running time** of the optimization algorithms for Michalewicz's function, **in seconds**

	Dimension = 5		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	0.84879	2.38903	1.17216
Hill Climbing: Best Improvement	0.81088	3.72689	1.41898
Hill Climbing: Worst Improvement	1.00298	3.11818	1.36494
Simulated Annealing	0.56492	3.97116	2.76521

	Dimension = 10		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	3.10319	7.0305	3.98309
Hill Climbing: Best Improvement	2.80705	14.98592	4.9537
Hill Climbing: Worst Improvement	3.64441	8.44909	5.00645
Simulated Annealing	0.00712	4.85428	2.07803

	Dimension = 30		
	Min. Value	**Max. Value**	**Mean**
Hill Climbing: First Improvement	27.17807	61.01697	35.3233
Hill Climbing: Best Improvement	28.14049	7801.40206	5.84932 min
Hill Climbing: Worst Improvement	43.05578	91.37614	60.14124
Simulated Annealing	11.06793	19.99679	14.13461

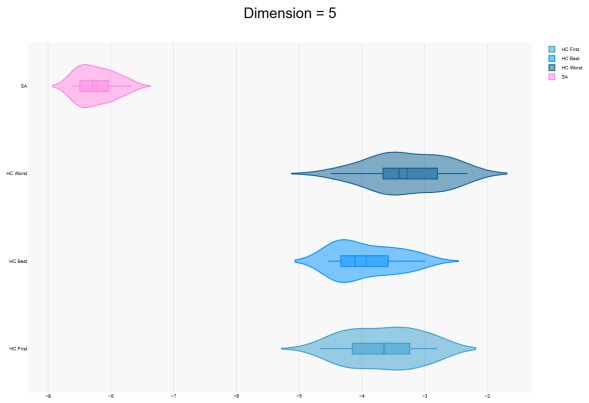

Figure 11: Comparison of optimization results on the Michalewicz function with dimension 5.

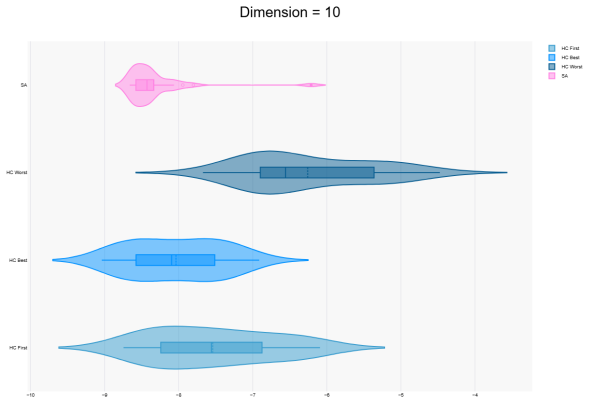

Figure 12: Comparison of optimization results on the Michalewicz function with dimension 10.

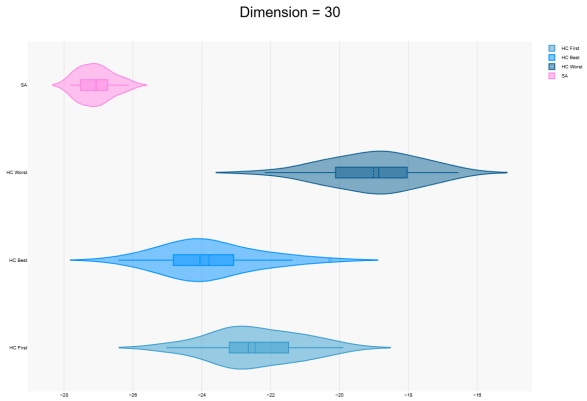

Figure 13: Comparison of optimization results on the Michalewicz function with dimension 30.

The plots for the Michalewicz's function show that all algorithms found good results, but Simulated Annealing performs the best overall, with more consistent values across executions. The Best Improvement version of Hill Climbing also does well, while the First and Worst versions show more variation between results. When the dimension increases, the results become more spread out, which means the problem gets harder. Still, both algorithms manage to find results close to the optimal value, most likely because the search space is smaller and easier to explore.

4 Conclusions

In this report, we unraveled the performance of two optimization algorithms, Hill Climbing and Simulated Annealing, on several benchmark functions, including De Jong's, Schwefel's, Rastrigin's, and Michalewicz's.The main goal was to see how each algorithm behaves under different levels of complexity and dimensionality and how effectively they can find the global minimum across various types of functions.
The results showed that Hill Climbing works very well on simpler and smoother functions, especially in lower dimensions, where its straightforward search quickly finds good solutions. However, its deterministic nature makes it vulnerable to local minima, especially as the complexity of the problem increases. Simulated Annealing proved to be more resilient under these conditions. By incorporating a controlled randomization process through its cooling schedule, it can explore the search space more broadly and escape local minima that typically trap Hill Climbing. Although this flexibility comes at the cost of higher computation time, it consistently achieved better solutions for complex and multimodal functions.
Overall, each algorithm demonstrates its own advantages. Hill Climbing excels in efficiently solving simpler optimization tasks, whereas Simulated Annealing performs better on complex problems where exploration is essential.

Resources and references

[1] Lect. dr. Eugen Croitoru. *Genetic Algorithms.*
Available at: https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/

[2] Daniel Delahaye, Supatcha Chaimatanan, Marcel Mongeau. *Simulated annealing: From basics to applications.*
Gendreau, Michel; Potvin, Jean-Yves. *Handbook of Metaheuristics*, 272, Springer, pp. 1–35.
ISBN 978-3-319-91085-7, 2019, International Series in Operations Research & Management Science (ISOR), 978-3-319-91086-4. ff10.1007/978-3-319-91086-4_1ff. ffhal-01887543f.

[3] *GEATBX: Examples of Objective Functions.*
Genetic and Evolutionary Algorithm Toolbox for use with Matlab. 1994-2000 Hartmut Pohlheim.
Available at: http://www.geatbx.com/docu/fcnindex-01.html .

[4] GeeksforGeeks. *Heuristic Search Techniques in AI.* Published July 23, 2025. Available at: https://www.geeksforgeeks.org/artificial-intelligence/heuristic-search-techniques-in-ai.

[5] GeeksforGeeks. *lt; iomanip gt; Header in C++.*. Published July 23, 2025. Available at: https://www.geeksforgeeks.org/cpp/iomanip-in-cpp. Accessed: October 2025.

[6] Doors of Stone. Daniel. *Measure elapsed time with chrono in C++.* (2023, March 7).
Available at : https://doorsofstone.hashnode.dev/measure-elapsed-time-with-chrono-lib-in-cpp .

[7] GeeksforGeeks. *std::mt19937 Class in C++.* (2021, March 30)
Available at: https://www.geeksforgeeks.org/cpp/stdmt19937-class-in-cpp/

[8] Stack Overflow. *Generating number (0,1) using Mersenne Twister c++.* (2014, April 7).
Available at: https://stackoverflow.com/questions/22923551/generating-number-0-1-using-mersenne-twister-c

[9] GeeksforGeeks. *Hill climbing in artificial intelligence.*(2025, August 1)
Available at: https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/

[10] cppreference.com, *std::uniform_real_distribution.* Available at: https://en.cppreference.com/w/cpp/numeric/random/uniform_real_distribution.html

[11] Deaken University. *Academic style writing.* Available at: https://www.deakin.edu.au/students/study-support/study-resources/study-support-guides/academic-style

[12] Marcin Molga, Czesław Smutnicki. *Test functions for optimization needs.* Wrocław University of Technology(3 april 2005)

[13] S. Ozdemir, *"The Mersenne Twister: A Reliable PRNG"* ,Medium, (2023 , October 3) Available at: https://medium.com/@ssserpilozdemir/the-mersenne-twister-a-reliable-prng-58a6eda64f64

[14] *Overview of optimization.* In Process systems engineering. (2006)(p. 285–314). Available at: https://doi.org/10.1016/s1874-5970(06)80012-3

[15] GeeksforGeeks. *Difference between hill climbing and simulated annealing algorithm.* (2025, July 23). Available at: https://www.geeksforgeeks.org/machine-learning/difference-between-hill-climbing-and-simulated-annealing-algorithm/

[16] S. Chen, S. Islam and S. Lao, *Simulated Annealing vs. Hill Climbing in Continuous Domain Search Spaces*, York University, Toronto, Canada, Apr. 2021. Available: https://www.yorku.ca/sychen/research/workshops/CEC2021_Workshop_on_Selection_SA.pdf

[17] Statistics Kingdom. *Violin plot maker.* Available at: https://www.statskingdom.com/violin-plot-maker.html